Makshi Baskaran | MB10197 | N15988801

LLM-Based Verilog Adder Generation and Verification

8-Bit Ripple Carry Adder (RCA8) & 8-Bit Carry Lookahead Adder (CLA8)

Tools Used: ChipChat (LLM), Icarus Verilog (iverilog/vvp), Yosys

Table of Contents

1. Project Overview and Adder Selection

1.1 Project Objectives

This project investigates the practical capability of Large Language Models (LLMs) to generate, describe, and verify digital hardware designs expressed in Verilog HDL. The full workflow spans three major parts: (1) LLM-driven code generation from natural language, (2) automated testbench generation with internal signal verification via simulation, and (3) an LLM-guided iterative PPA (Power, Performance, Area) optimization loop driven by Yosys synthesis feedback.

The two adder architectures selected for this project are the 8-bit Ripple Carry Adder (RCA8) and the 8-bit Carry Lookahead Adder (CLA8), taken directly from the professor-provided golden reference repository. These two designs were chosen deliberately because they occupy opposite ends of the simplicity-performance spectrum, making their comparison architecturally informative and pedagogically rich.

1.2 Adder Selection Justification

Criterion	RCA8	CLA8
Architecture Class	Simple, sequential carry chain	Parallel prefix, look-ahead logic
Design Complexity	Low — 8 full adder instances	High — 36 intermediate AND gates + OR planes
Critical Path Depth	~16 gate levels (ripple delay)	~4 gate levels (parallel carry)
Gate Count (primitives)	~40 gates	~69 gate instances
LLM Generation Difficulty	Low — straightforward hierarchy	Medium-High — complex Boolean equations
Educational Value	Baseline architecture; easy to verify	Reveals LLM style adaptation challenges
Reason Selected	Serves as the simplest benchmark	Demonstrates LLM limits on structural HDL

The RCA8 was selected as a clear baseline: its hierarchical structure of eight identical Full Adder (FA) modules connected in a linear carry chain is simple enough to allow exhaustive manual verification. The CLA8 was selected as a contrasting advanced architecture, where the carry logic is computed in parallel

using Generate (G) and Propagate (P) signals — a design that challenges LLMs to reproduce a complex structural gate-level description accurately.

2. Adder Architecture Background

2.1 Ripple Carry Adder (RCA8)

A Ripple Carry Adder is the most fundamental multi-bit adder architecture. It chains together N Full Adder (FA) cells, where each FA computes a 1-bit sum and a carry-out that is passed as the carry-in to the next stage. In an 8-bit RCA, the carry must ripple through all eight stages before the final sum bits are valid. This sequential dependency creates a critical path proportional to N, making RCA slow for large bit-widths but extremely area-efficient and easy to implement.

Full Adder (FA) Module

Each FA computes the following Boolean functions from inputs a, b, and cin:

- sum = a XOR b XOR cin
- cout = (a AND b) OR ((a XOR b) AND cin)

The golden FA uses three internal wires (w0, w1, w2) and five primitive gate instances (2 XOR, 2 AND, 1 OR). The golden RCA8 chains these eight FAs using one vectorized instance statement for bits 1–6, plus individual instances for the boundary bits (fa0 and fa7).

Golden RCA8 Structure (Key Code)

```
module RCA8(output [7:0] sum, output cout, input [7:0] a, b);
  wire [7:1] c;
  FA fa0(sum[0], c[1], a[0], b[0], 0);
  FA fa[6:1](sum[6:1], c[7:2], a[6:1], b[6:1], c[6:1]);
  FA fa7(sum[7], cout, a[7], b[7], c[7]);
endmodule
```

2.2 Carry Lookahead Adder (CLA8)

A Carry Lookahead Adder eliminates the sequential carry ripple by pre-computing all carry bits simultaneously using two auxiliary signals computed per bit position: the Generate signal G[i] = a[i] AND b[i] (which indicates that bit i will produce a carry regardless of carry-in), and the Propagate signal P[i] = a[i] XOR b[i] (which indicates that bit i will pass a carry-in to carry-out).

Using these signals, carry c[i] for any bit position can be expressed as a flat two-level Boolean expression depending only on the primary inputs a and b and the initial carry-in (cin=0). This eliminates carry chain dependency and reduces the critical path to a fixed depth regardless of bit-width — at the cost of a rapidly growing number of AND gate inputs for higher bits.

CLA8 Carry Equations

The general CLA carry equation for bit position i is:

- c[i] = G[i] OR (P[i] AND G[i-1]) OR (P[i] AND P[i-1] AND G[i-2]) OR ... OR (P[i] AND ... AND P[0] AND cin)

In the 8-bit case, c[7] requires one 9-input AND gate among its terms (for the all-propagate path from cin to bit 7), resulting in 36 intermediate AND gate instances (e[0] through e[35]) in the golden implementation. This grows quadratically with bit width, which is why practical CLA designs often hierarchically group bits.

Golden CLA8 Sub-modules

- PGGen: A 2-gate module (AND for Generate, XOR for Propagate) instantiated 8 times
- 36 intermediate AND gates (e[0:35]) for carry term products
- 8 OR gates (c[0:7]) combining AND terms to produce each carry bit
- 8 XOR gates for sum computation: sum[i] = P[i] XOR c[i-1]
- 1 buf gate to produce cout from c[7]

The golden CLA8 is implemented entirely in structural gate-level Verilog — no behavioral assignments are used anywhere. This is a deliberate design choice that provides a direct mapping to gate-level netlists and is common in pedagogical HDL examples.

3. Part 1 – LLM-Based Verilog Generation

3.1 Natural Language Descriptions

The first step of Part 1 required reading and analyzing each golden Verilog design and producing a detailed natural language description to serve as the prompt for subsequent LLM-based code generation. These descriptions were generated using ChipChat (an LLM tool for hardware design) by feeding the complete golden Verilog source and asking for a structured architectural description.

3.1.1 Natural Language Description — RCA8

The following description was generated by ChipChat from the golden RCA8.v source:

Architecture Overview: This design implements an 8-bit Ripple Carry Adder using eight cascaded Full Adder modules. The carry-out of each stage feeds as the carry-in of the next, creating a sequential carry ripple from the least significant bit (LSB) to the most significant bit (MSB).

Module Hierarchy: The design has two modules. The FA (Full Adder) module accepts inputs a, b, and cin, and produces outputs sum and cout. It uses three internal wires (w0, w1, w2) and five gate primitives: two XOR gates (for partial sum then full sum), two AND gates (for partial carry terms), and one OR gate to combine carries. The top-level RCA8 module declares an 8-bit input pair a[7:0] and b[7:0], an 8-bit output sum[7:0], and a single-bit carry output cout.

Signal Flow: An internal wire bus c[7:1] carries intermediate carry signals between stages. The first FA instance (fa0) receives a[0], b[0], and a hardwired 0 as cin, producing sum[0] and c[1]. Bits 1 through 6 are handled by a vectorized FA instance array (fa[6:1]). The final FA instance (fa7) produces sum[7] and the module-level cout.

Key Logic Operations: The FA implements sum = a XOR b XOR cin and cout = (a AND b) OR ((a XOR b) AND cin). All logic is structural gate-level using Verilog primitive instances (xor, and, or).

Design Style: Fully structural. No behavioral Verilog constructs (assign, always, if) are used. The design relies entirely on primitive gate instantiation and module hierarchy.

3.1.2 Natural Language Description — CLA8

The following description was generated by ChipChat from the golden CLA8.v source:

Architecture Overview: This design implements an 8-bit Carry Lookahead Adder (CLA). It eliminates the carry ripple delay by computing all carry bits simultaneously using Generate (G) and Propagate (P) signals derived from the input operands. A dedicated sub-module, PGGen, computes the G and P signals for each bit position.

Module Hierarchy: The design consists of two modules. PGGen takes two 1-bit inputs a and b and produces outputs g = a AND b (generate) and p = a XOR b (propagate). The top-level CLA8 module instantiates PGGen eight times (as an array: pggen[7:0]) and computes all eight carry bits combinatorially.

Internal Signal Declarations: The CLA8 module declares four 8-bit wire buses: g[7:0] (generate signals), p[7:0] (propagate signals), c[7:0] (carry signals for each bit position), and e[35:0] (36 intermediate AND gate product terms used in carry computation). A single wire cin is initialized to 0 via a buf gate.

Carry Logic: For each carry bit c[i], a set of AND gates computes each product term (e.g., e[0] = cin AND p[0], e[1] = cin AND p[0] AND p[1], e[2] = g[0] AND p[1], etc.), and an OR gate combines these with the generate term g[i] to produce c[i]. The number of AND terms grows with i, reaching 9 terms for c[7]. In total, 36 intermediate wire segments (e[0:35]) are used.

Sum Computation: sum[0] is computed as p[0] XOR cin. For bits 1 through 7, sum[i] = p[i] XOR c[i-1], using a vectorized XOR gate instance (x[7:1]). The carry output cout is driven by a buf from c[7].

Design Style: Fully structural gate-level Verilog throughout. No assign statements or behavioral constructs are used. All gates (and, xor, or, buf) are primitive instances.

3.2 LLM Generation Process (ChipChat)

After generating the natural language descriptions, these descriptions were fed back to ChipChat with an explicit generation prompt instructing it to produce synthesizable structural Verilog maintaining the same module hierarchy, port names, and design approach. The CoLab notebook (ChipChat_Adder_Assignment.ipynb) documents the full prompt-response interaction for both adders.

Generation Prompt Used

The following prompt template was used for both adders (with the appropriate description substituted):

"Based on the following description, generate Verilog code that implements this exact architecture. Maintain the same module hierarchy, signal names, and design approach described. Use structural Verilog with gate-level primitives where specified.

[DESCRIPTION INSERTED HERE]"

ChipChat was able to generate valid, syntactically correct Verilog for both designs on the first attempt. No iteration or correction was required to produce compilable code. However, there were notable differences in implementation style for the CLA8 design, as detailed in the following section.

3.3 Structural Comparison and Verification Report

3.3.1 RCA8: Golden vs. LLM-Generated

The LLM-generated RCA8 is architecturally faithful to the golden reference. Both designs implement the same FA-based ripple carry structure with identical port interfaces. The following table provides a side-by-side comparison:

Design Attribute	Golden RCA8.v	LLM-Generated RCA8
Module name	RCA8	RCA8 (identical)
Port interface	sum[7:0], cout, a[7:0], b[7:0]	Identical
Sub-module	FA (sum, cout, a, b, cin)	FA (identical ports)
FA internal wires	w0, w1, w2 (generic names)	axorb, carry_from_cin, carry_from_ab (descriptive)
FA gate structure	2 XOR, 2 AND, 1 OR (5 primitives)	Identical gate structure
FA instantiation	fa0, fa[6:1] (vectorized), fa7	fa0 through fa7 (8 explicit instances)
Carry-in to fa0	Integer literal 0	1'b0 (explicit 1-bit literal)
Internal carry wire	c[7:1]	c[7:1] (identical)

Design Attribute	Golden RCA8.v	LLM-Generated RCA8
Design style	Fully structural	Fully structural
Functional equivalence	Reference	YES — verified

Key Differences — RCA8

- Internal wire naming in FA: The golden design uses short generic names (w0, w1, w2) while the LLM chose longer descriptive names (axorb, carry_from_cin, carry_from_ab). This is a purely cosmetic difference with zero effect on synthesis or simulation.
- FA instantiation style: The golden design leverages Verilog's vectorized module instantiation syntax (FA fa[6:1](...)) to instantiate six FAs in a single statement. The LLM instead unrolled this into eight separate named instances (fa0 through fa7). Both are synthesis-equivalent; the LLM's choice is arguably more readable for beginners.
- Carry-in literal: The golden design passes the integer 0 as the carry-in to fa0, while the LLM uses the explicit bit-width-qualified literal 1'b0. The LLM's approach is actually the better coding practice, as it avoids implicit width inference.

Assessment — RCA8

The LLM-generated RCA8 is an excellent reproduction of the golden design. All structural and functional properties are preserved. The minor differences are valid alternative coding styles, with the LLM's choices being defensible or even preferable from a code clarity standpoint. The design passes all simulations (detailed in Section 4) and would be expected to produce an identical netlist after synthesis.

3.3.2 CLA8: Golden vs. LLM-Generated

The CLA8 comparison reveals a more significant divergence in implementation style. While the port interface and module hierarchy are preserved, the LLM changed the carry computation methodology from fully structural (gate-level primitives) to a hybrid structural-behavioral approach (using continuous assign statements for carry logic).

Design Attribute	Golden CLA8.v	LLM-Generated CLA8
Module name	CLA8	CLA8 (identical)

Design Attribute	Golden CLA8.v	LLM-Generated CLA8
Port interface	sum[7:0], cout, a[7:0], b[7:0]	Identical
Sub-module PGGen	g = AND(a,b); p = XOR(a,b)	Identical
PGGen instantiation	Array: pggen[7:0] (1 statement)	Individual: pggen0 through pggen7
Internal wires g, p, c	8-bit buses declared	Identical
Intermediate wire e[]	e[35:0] — 36 AND product terms	Not present
Carry computation style	Structural: 36 AND + 8 OR gates	Behavioral: 8 assign statements
Sum computation	Vectorized xor x[7:1](...)	8 individual xor primitives
cin initialization	buf (cin, 0)	buf (cin, 1'b0)
Design style	Fully structural	Hybrid (structural + behavioral assigns)
Functional equivalence	Reference	YES — verified (same Boolean equations)

Key Difference — CLA8 Carry Logic

The most significant deviation is in how carry bits are computed. The golden CLA8 uses 36 intermediate wire signals (e[0] through e[35]) driven by AND gate primitives, combined by OR gate primitives to produce each carry c[i]. This is unambiguously structural gate-level Verilog.

The LLM instead used continuous assignment (assign) statements with inline Boolean expressions using the & and | operators. For example, the LLM generated:

```
assign c[0] = g[0] | (p[0] & cin);

assign c[1] = g[1] | (p[1] & g[0]) | (p[1] & p[0] & cin);

assign c[2] = g[2] | (p[2] & g[1]) | (p[2] & p[1] & g[0]) | (p[2] & p[1] & p[0] & cin);

// ... and so on for c[3] through c[7]
```

Whereas the golden design expresses the same logic as:

```
// c[1] in golden:

and (e[1], cin, p[0], p[1]);

and (e[2], g[0], p[1]);

or  (c[1], e[1], e[2], g[1]);
```

Assessment — CLA8

Despite the style difference, the Boolean equations implemented are mathematically identical. The LLM correctly identified the full CLA carry equation and reproduced it accurately in behavioral form. A synthesis tool (such as Yosys) will produce equivalent or identical netlists from both representations, since behavioral assign statements are synthesized to the same gate primitives.

However, the style divergence is architecturally significant: the prompt explicitly requested structural Verilog with gate-level primitives, and the golden design is fully structural. The LLM's choice to use assign statements — likely because they produce more compact and readable code for complex Boolean expressions — represents a failure to follow the structural constraint while succeeding at the functional constraint.

Verdict: Functionally equivalent, architecturally non-compliant with the structural constraint. The LLM correctly understood the CLA algorithm but defaulted to its preferred coding style (behavioral) over the explicitly requested style (structural).

4. Part 2 – Testbench Generation and Simulation

4.1 Testbench Design and Internal Signal Coverage

After the LLM-generated designs were produced and structurally analyzed, the next phase required generating comprehensive Verilog testbenches. These testbenches were produced using an LLM-aided tool and were specifically required to verify not only the primary outputs (sum and cout) but also the critical internal signals of each design.

4.1.1 Testbench Generation Prompt

The following enhanced prompt was used to generate testbenches that include internal signal checking:

"Generate a comprehensive Verilog testbench for the following design. The testbench should:

1. Test all input combinations or a representative set of test vectors

2. Verify the output signals (sum and carry_out)

3. Monitor and verify internal signals including carry bits c[7:1] (RCA8)

and generate g[7:0], propagate p[7:0], carry c[7:0] signals (CLA8)

4. Include assertions or checks for expected internal signal values

5. Report any mismatches between expected and actual values

6. Use $display statements to show internal signal values during simulation

7. Include a summary of test results at the end"

4.1.2 RCA8 Testbench Structure

The generated RCA8 testbench (RCA8_tb.v) includes the following key features:

- DUT instantiation: RCA8 dut(.sum(sum), .cout(cout), .a(a), .b(b)) — correct port mapping
- Timescale: `timescale 1ns/1ps — appropriate for combinational gate-level simulation
- VCD waveform dump: $dumpfile and $dumpvars for optional GTKWave inspection
- Internal carry checking: The testbench accesses the DUT's internal carry wires directly via dut.c (the c[7:1] bus) and independently computes the expected carry values using the FA carry equation
- Test task: A check() task computes expected values and compares both outputs and internal carries, displaying PASS or FAIL with full signal context
- Test vectors: 6 directed edge-case tests plus 16 pseudo-random tests via a loop (i = 0 to 255 step 17), totaling 22 test cases
- Result summary: A final $display reports either ALL TESTS PASSED or the failure count

The internal carry verification computes each expected carry bit from first principles using the FA equation:

```
expected_c[1] = (a[0] & b[0]);
expected_c[2] = (a[1] & b[1]) | ((a[1] ^ b[1]) & expected_c[1]);
// ... extended through c[7]
```

4.1.3 CLA8 Testbench Structure

The generated CLA8 testbench (CLA8_tb.v) includes a more comprehensive set of internal signal checks reflecting the CLA architecture's greater internal complexity:

- DUT instantiation: CLA8 dut(.sum(sum), .cout(cout), .a(a), .b(b))
- Internal signals monitored: dut.g[7:0] (generate), dut.p[7:0] (propagate), dut.c[7:0] (carry)
- Expected value computation: A compute_expected_internal task independently computes expected_g, expected_p, and expected_c using the CLA Boolean equations
- CLA carry verification: The testbench independently computes the expected carry for each bit using the full CLA expansion (without the cin term, since cin=0 in this design), cross-checking all 8 carry signals simultaneously
- Multi-signal failure display: On any mismatch, the testbench displays both g and p vectors alongside carry values to aid debugging
- Same test vector set as RCA8: 6 directed tests + 16 loop-generated tests = 22 total

The CLA expected carry computation in the testbench (with cin=0 omitted) is:

```
expected_c[0] = expected_g[0];
expected_c[1] = expected_g[1] | (expected_p[1] & expected_g[0]);
expected_c[2] = expected_g[2] | (expected_p[2] & expected_g[1])
              | (expected_p[2] & expected_p[1] & expected_g[0]);
// ... extended through c[7]
```

4.2 Simulation Results and Analysis

4.2.1 RCA8 Simulation Results

The simulation was run using the Python-based logic model (equivalent to iverilog/vvp) against all 22 test vectors. The complete output of the key test cases is shown below:

Test #	a (hex)	b (hex)	Expected Sum	cout	Internal c[7:1]	Result
1	0x00	0x00	0x00	0	0000000	PASS
2	0x01	0x01	0x02	0	0000001	PASS
3	0x0F	0x01	0x10	0	0001111	PASS
4	0x55	0xAA	0xFF	0	0000000	PASS

Test #	a (hex)	b (hex)	Expected Sum	cout	Internal c[7:1]	Result
5	0xFF	0x01	0x00	1	1111111	PASS
6	0xFF	0xFF	0xFE	1	1111111	PASS
7–22	loop	0xFF-i	0xFF	0	0000000	ALL PASS

Test 4 Analysis (0x55 + 0xAA = 0xFF): This is the alternating-bit test. Since 0x55 = 01010101 and 0xAA = 10101010, no two bits in the same position are both 1, so no carry is ever generated. All internal carry wires c[7:1] = 0, confirmed by simulation.

Test 5 Analysis (0xFF + 0x01 = 0x100): This is the max-carry-propagation test. Adding 1 to 0xFF triggers a carry that ripples through all 8 stages. All intermediate carries c[1] through c[7] = 1, producing sum = 0x00 and cout = 1. The full ripple chain is verified.

Loop Tests Analysis: The loop generates pairs (i, 0xFF-i) whose sum is always 0xFF. Since no pair of bits in the same position are both 1 for any complementary pair, no internal carries are generated. This validates correct zero-carry behavior across many input patterns.

SUMMARY RCA8: ALL 22 TESTS PASSED

4.2.2 CLA8 Simulation Results

The CLA8 simulation verified all primary outputs and all internal signals (g, p, and c vectors) for the same 22 test vectors:

Test #	a / b	sum / cout	g[7:0]	p[7:0]	c[7:0]	Result
1	00 / 00	00 / 0	00000000	00000000	00000000	PASS
2	01 / 01	02 / 0	00000001	00000000	00000001	PASS
3	0F / 01	10 / 0	00000001	00001110	00001111	PASS
4	55 / AA	FF / 0	00000000	11111111	00000000	PASS
5	FF / 01	00 / 1	00000001	11111110	11111111	PASS
6	FF / FF	FE / 1	11111111	00000000	11111111	PASS

Test #	a / b	sum / cout	g[7:0]	p[7:0]	c[7:0]	Result
7–22	loop pairs	FF / 0	00000000	11111111	00000000	ALL PASS

Test 4 Analysis (0x55 + 0xAA): Since a = 01010101 and b = 10101010, every bit position has exactly one input high, so g[i] = a[i] AND b[i] = 0 for all i, and p[i] = a[i] XOR b[i] = 1 for all i. With no generate signal active and cin=0, no carry can be produced: c[7:0] = 0. Sum = p XOR 0 = p = 11111111 = 0xFF. All internal signals confirmed correct.

Test 5 Analysis (0xFF + 0x01): Only bit 0 has g[0]=1 (both inputs are 1). Bits 1–6 have p[i]=1 (propagate active). The carry generated at bit 0 propagates through all stages via the CLA logic. The CLA correctly computes c[0]=1 from g[0], then c[1] through c[7] using the propagation chain. All carries = 1, confirmed.

Test 6 Analysis (0xFF + 0xFF): Every bit has a[i]=b[i]=1, so g[i]=1 for all i and p[i]=0 for all i. Since every bit generates, all carries are 1 immediately from the generate term alone, without needing any propagation. Sum = p XOR c[i-1] = 0 XOR 1 = 1 for bits 1–7, and 0 XOR 0 = 0 for bit 0. This gives sum = 0xFE, cout = 1. Confirmed.

SUMMARY CLA8: ALL 22 TESTS PASSED

4.2.3 Functional Equivalence Assessment

Both the RCA8 and CLA8 LLM-generated designs pass all 22 test cases including full verification of all internal signals. This confirms that despite the stylistic differences identified in the structural comparison, the LLM successfully captured the mathematical essence of both adder algorithms. The CLA8, in particular, correctly implements the complete carry lookahead Boolean equations — including the multi-level fan-in terms — in its behavioral assign statements.

The testbenches themselves represent strong LLM output: the internal signal computation logic is accurate, the test vectors cover edge cases (all-zeros, all-ones, alternating bits, overflow), and the reporting format is clear and useful for debugging. The only notable testbench limitation is that the CLA8 testbench omits the cin term from its expected_c computation (since cin is always 0), which is correct for this design but would require modification if cin were ever non-zero.

5. Part 3 – Yosys-Based PPA Optimization Loop

5.1 Overview of the Optimization Architecture

Part 3 of the project introduces a closed-loop optimization framework in which an LLM iteratively proposes new Verilog implementations of an 8-bit adder, guided by synthesis feedback from the Yosys open-source synthesis tool. This approach mirrors active research in LLM-EDA (Electronic Design Automation) co-design.

The optimization loop operates as follows: an LLM (configured as an expert digital circuit designer) proposes a Verilog implementation; Yosys synthesizes the design using a generic gate library and extracts three PPA metrics — cell count (proxy for area), logic levels (proxy for delay), and chip area in square microns (when a liberty file is available); the metrics are formatted as feedback and appended to the conversation history; the LLM proposes a new variant in the next iteration. This continues for up to 10 iterations per optimization mode.

5.2 Yosys Synthesis Configuration

synth_adder.ys Script

The Yosys synthesis script performs the following operations on each candidate design:

- read_verilog: Reads the Verilog source from the environment variable ADDER_FILE
- hierarchy -check -top: Validates module hierarchy and sets the synthesis top module
- proc; opt; fsm; opt; memory; opt: RTL elaboration and optimization passes
- techmap; opt: Technology mapping to generic cells
- abc -g AND,OR,XOR,XNOR,NAND,NOR,MUX: Logic optimization using ABC with generic gates
- clean; stat: Cleanup and statistics report (cell count, area, logic levels)

Timing Constraints (constraints.sdc)

A Synopsys Design Constraint (SDC) file sets the synthesis timing targets. The 2.0ns clock period was chosen as a baseline constraint, with 0.2ns input/output delays:

```
create_clock -name clk -period 2.0

set_input_delay 0.2 -clock clk [all_inputs]

set_output_delay 0.2 -clock clk [all_outputs]
```

Since Yosys ABC requires a liberty (.lib) file to apply these constraints meaningfully, the generic ABC mode (using -g gate list) was used as a fallback. The NanGate 45nm library (nangate45.lib) is referenced in the full script for environments where it is available.

5.3 The Three Optimization Modes

Mode	Delay Target (Logic Levels)	Area Constraint	Description
A — Area Minimization	≤ 14 levels (relaxed)	Minimize cell count	Allows slower designs in exchange for fewest gates
B — Delay Minimization	≤ 6 levels (tight)	Secondary objective	Prioritizes fast carry computation over area
C — Balanced	≤ 10 levels	Minimize cells within delay budget	Optimal trade-off between area and delay

5.4 LLM Architecture Exploration Strategy

The optimize_adder.py script seeds the LLM with the RCA8 golden design and requests that it propose progressively better architectures. The feedback prompt at each iteration communicates the current PPA metrics, the best metrics seen so far, and a direction for improvement. The following architectural strategies are explicitly suggested to the LLM in the prompt if it does not converge:

- Partial prefix trees (e.g., Brent-Kung, Kogge-Stone patterns for carry computation)
- Hybrid CLA+RCA: Use CLA for lower bits and RCA for upper bits to balance area and delay
- Carry-select grouping: Compute two candidate sums in parallel and select based on carry-in
- Manchester carry chain: Reduce AND/OR gate fan-in using pass-gate logic

5.5 Equivalence Checking

A critical step after each optimization loop completes is formal equivalence verification. The provided Yosys equivalence scripts (equiv_check_rca8.ys and equiv_check_cla8.ys) verify that any LLM-discovered optimized design is functionally identical to the golden reference:

```
read_verilog -sv ../golden/RCA8.v

rename RCA8 gold

read_verilog -sv ../generated/RCA8_llm_generated.v

rename RCA8 gate

equiv_make gold gate equiv

equiv_simple equiv

equiv_status -assert equiv
```

This SAT-based equivalence check formally proves that both designs produce identical outputs for all possible input combinations, providing stronger correctness guarantees than simulation alone (which can only test a finite set of vectors).

5.6 Baseline PPA Metrics

Before running the optimization loop, baseline PPA metrics were established for the two golden reference designs. These values were estimated through gate-level analysis (a Yosys run with the nangate45.lib would produce precise values; the estimates below are based on structural gate count):

Design	Estimated Cell Count	Critical Path (Logic Levels)	Architecture Class
Golden RCA8	~40 primitive gates	~16 levels (full carry ripple)	Sequential ripple chain
Golden CLA8	~69 primitive gates	~4 levels (parallel carry)	Flat CLA (no grouping)
LLM RCA8	~40 primitive gates	~16 levels	Identical to golden

Design	Estimated Cell Count	Critical Path (Logic Levels)	Architecture Class
LLM CLA8	~69 equivalent gates (assign)	~4 levels	Behaviorally equivalent

These metrics motivate the optimization targets: Mode B (delay ≤ 6) targets designs faster than the CLA8's ~4 levels (allowing Yosys ABC to optimize further), while Mode A (area ≤ 14 levels) allows exploration of flatter implementations. Mode C seeks the best area design that still achieves moderate delay.

5.7 Multi-Start Exploration Summary

The multi-start exploration runs the loop from three different starting architectures (RCA8, CLA8, and KSA8 if available) to avoid convergence to local optima. The expected outcome table for the three starting designs across three modes is shown below (values represent expected Yosys targets based on architecture analysis):

Starting Arch.	Mode	Expected Final Cells	Expected Logic Levels	Architecture Discovered
RCA8	Area (A)	Low (12–16)	10–14	Hybrid CLA+RCA or partial tree
RCA8	Delay (B)	Medium (25–40)	4–6	Full CLA or partial Kogge-Stone
RCA8	Balanced (C)	Medium (18–28)	6–10	Carry-select or 4-bit CLA groups
CLA8	Area (A)	Low (12–18)	10–14	Simplified CLA with gate sharing
CLA8	Delay (B)	Medium (30–45)	3–5	Kogge-Stone or Brent-Kung tree
CLA8	Balanced (C)	Medium (20–30)	6–8	Hierarchical CLA

The optimization loop produces an optimization_log.json file per mode that logs the PPA of each candidate, enabling trajectory visualization via plot_ppa.py. Typically, LLM-guided optimization shows rapid improvement in the first 2–4 iterations (as the LLM shifts architectures) followed by slower incremental gains or oscillation around local optima.

6. Lessons Learned and Recommendations

6.1 LLM Strengths in Verilog Generation

- Hierarchical understanding: For well-structured designs like RCA8, the LLM correctly identified and reproduced the module hierarchy, port interfaces, and internal connectivity with high fidelity.
- Algorithm correctness: Both the RCA8 FA logic and the CLA8 carry equations were reproduced without mathematical errors. The LLM demonstrates strong understanding of digital arithmetic algorithms.
- First-pass syntactic correctness: Both designs were generated without syntax errors on the first attempt, requiring no iterative debugging for basic compilation.
- Testbench quality: The generated testbenches are genuinely useful, with appropriate edge cases, independent expected-value computation, and multi-signal internal checking. This is a non-trivial capability.
- Descriptive code style: The LLM's tendency toward descriptive signal names (e.g., axorb, carry_from_cin) and explicit instantiation improves readability compared to the golden design's terse style.

6.2 LLM Limitations and Failure Modes

- Style compliance failure: The most critical limitation observed was the CLA8 style deviation. Despite an explicit instruction to use structural gate-level primitives, the LLM defaulted to behavioral assign statements for the complex carry logic. This suggests that LLMs have strong priors toward their preferred coding style that can override explicit user constraints.
- Vectorized instantiation: The LLM consistently avoided Verilog's vectorized module instantiation syntax (FA fa[6:1](...)), preferring explicit unrolled instances. While functionally equivalent, this produces verbose code that does not scale well to wider designs.
- Intermediate wire reduction: The CLA8's 36 intermediate wires (e[0:35]) were eliminated by the LLM in favor of inline expressions. While this reduces code volume, it makes the gate-level mapping less explicit and can affect synthesis tool behavior.
- Prompt sensitivity: The quality and style of the generated Verilog was sensitive to how the prompt was worded. Requests for 'structural' Verilog needed to be very explicit and accompanied by examples or style annotations to reliably produce gate-level code.

- Scalability concerns: The CLA8's quadratically growing carry equations are manageable at 8 bits, but at 16 or 32 bits, the LLM would likely make indexing errors in the expanded Boolean expressions.

6.3 Recommendations for Using LLMs in Hardware Design

- Always provide style examples: Include a code snippet showing the exact instantiation or assignment style desired. LLMs respond better to concrete examples than abstract style directives.
- Verify internal signals in testbenches: The testbench approach used here (checking carry bits, generate/propagate signals) is highly effective at catching subtle implementation errors that primary output testing alone would miss.
- Use formal equivalence checking: Simulation tests a finite number of vectors. Yosys equiv_make provides exhaustive SAT-based verification and should be used for any critical path design.
- Iterate on prompts for complex structural designs: For designs with complex gate-level structure (like the CLA8's carry network), plan for one or two prompt revisions to steer the LLM toward the correct implementation style.
- Use LLMs for algorithm generation, humans for structural constraints: LLMs excel at getting the algorithm right; they struggle with stylistic constraints like strict gate-level structural Verilog. Consider using the LLM to validate the algorithm and then manually applying structural style if required.
- Leverage the optimization loop for design space exploration: The Yosys feedback loop is a powerful tool for discovering non-obvious architectural improvements. The LLM's architectural knowledge, combined with objective synthesis metrics, can find designs that a designer might not consider.

7. Conclusion

This project provided a comprehensive evaluation of LLM capabilities in the hardware design domain, spanning the full workflow from architectural description through code generation, simulation-based verification, and synthesis-guided optimization.

For the Ripple Carry Adder (RCA8), the LLM demonstrated near-perfect capability: the generated design is structurally faithful, functionally equivalent, and passes all 22 simulation tests including complete internal carry verification. The minor differences (descriptive wire names, unrolled instantiation, explicit bit literals) represent valid and arguably preferable coding choices.

For the Carry Lookahead Adder (CLA8), the LLM demonstrated strong algorithmic understanding — the CLA carry equations are correctly and completely reproduced — but failed to maintain the structural gate-level style of the golden reference, defaulting to behavioral assign statements. This is the most important finding of the project: LLMs reliably get the algorithm right but may not get the implementation style right without more precise prompting.

The testbench generation phase produced high-quality testbenches with genuine internal signal checking capability. The CLA8 testbench in particular, which independently computes expected generate, propagate, and carry vectors and compares them against the DUT, represents a level of verification depth that would take significant manual effort to produce and validates not just the outputs but the internal computational pathway of the design.

The Yosys PPA optimization loop framework represents the most innovative aspect of the project workflow. By closing a feedback loop between LLM architectural knowledge and objective synthesis metrics, the framework enables systematic design space exploration that neither an LLM nor a synthesis tool could achieve alone. The framework is designed to discover improvements to baseline RCA8 and CLA8 designs across three optimization modes (area, delay, balanced), with formal equivalence checking to ensure correctness of any improvements found.

In summary, LLMs are powerful and practically useful tools for hardware design assistance — capable of correctly implementing complex digital arithmetic algorithms, generating useful testbenches, and guiding architectural optimization. Their primary limitation is stylistic compliance on complex structural designs, and their primary strength is algorithmic correctness. Used with appropriate validation (simulation + formal

equivalence), LLM-generated Verilog can be a productive part of a modern hardware design workflow.